University of West Bohemia / FAV

Seminar Project - Artificial Intelligence

Document Classification System

Analysis and Implementation using the Reuters-21578 Corpus

Rodrigo Portugal Hortigüela

Total time of SP solution: 50 hours
Submission date: May 8, 2026

May 8, 2026

Contents

1 Description of the Problem **2**

2 Problem Analysis **2**

3 Suggestion of Solution **2**

4 Solution Description **2**
4.1 Classifiers and Mathematics 2

5 User Documentation **3**
5.1 CLI Mode: Training and Evaluation 3
5.2 GUI Mode: Inference 3
5.3 Bonus Mode: Scikit-Learn Comparison 3

6 Conclusion **4**

1 Description of the Problem

The main goal of this seminar project was to design and implement an Artificial Intelligence program capable of classifying text documents into distinct categories based on their semantic content. The dataset selected for this task is the **Reuters-21578 corpus**. The problem specification strictly required the implementation of three different parameterization algorithms to convert raw text into mathematical feature vectors, alongside three distinct classification algorithms to categorize the documents.

2 Problem Analysis

Raw text cannot be directly interpreted by mathematical machine learning algorithms. This process, known as parameterization or vectorization, faces challenges such as the sheer volume of unique words and the presence of highly frequent but semantically meaningless words (noise).

Additionally, different classification algorithms react differently to how data is scaled. Geometric-based algorithms can be heavily distorted by large word counts, while probabilistic algorithms require specific handling (like smoothing) to avoid zero-probability errors.

3 Suggestion of Solution

The system utilizes a pipeline that processes the Reuters corpus, extracts features, and feeds them into the classifiers.

For parameterization, the following algorithms were selected:

- **Binary:** A simple indicator of word presence.
- **Term Frequency (TF):** Counts raw occurrences.
- **Term Frequency - Inverse Document Frequency (TF-IDF):** Weighs the term frequency against the rarity of the word across the entire corpus.

$$\text{TF-IDF}(t, d, D) = \text{tf}(t, d) \cdot \log\left(\frac{N}{|\{d \in D : t \in d\}|}\right) \tag{1}$$

4 Solution Description

The implemented solution is a comprehensive system that analyzes the `.sgm` files from the Reuters corpus. The system utilizes specific fields to divide the dataset and extracts the first tag from the documents to assign the ground-truth category. The raw text is then transformed into feature vectors using the implemented parameterization algorithms.

4.1 Classifiers and Mathematics

The chosen primary algorithm is **K-Nearest Neighbors (KNN)** with $k = 5$. For geometric classifiers like **Minimum Distance**, the distance d between a document x

and a class centroid μ is calculated using the Euclidean Distance:

$$d(x, \mu) = \sqrt{\sum_{i=1}^{n}(x_i - \mu_i)^2} \tag{2}$$

The implementation also includes **Naive Bayes**, which uses probabilistic log-likelihoods to prevent numerical underflow.

5 User Documentation

The program features a dual interface for maximum flexibility.

5.1 CLI Mode: Training and Evaluation

Run the following in PowerShell to train a model:

```
Python main.py <training_data_path> <test_data_path> <parametrization>
<classifier> <model_name>
```

Example: Training a Naive Bayes model using TF-IDF:

```
py main.py data/reuters21578 data/reuters21578 tfidf naive_bayes
my_model_nb_tfidf
```

5.2 GUI Mode: Inference

To launch the graphical interface with a pre-trained model:

```
py main.py my_model_nb_tfidf
```

Testing the GUI: You can input the following text (related to acquisitions) to verify the prediction (`acq.`):

> *"The technology giant announced today that it has agreed to acquire the smaller software startup for 500 million dollars in cash. The merger and takeover process is expected to close next month."*

5.3 Bonus Mode: Scikit-Learn Comparison

To compare the manual implementation with the industry-standard library:

```
py bonus_comparison.py data/reuters21578
```

6 Conclusion

The main goal of the problem was successfully achieved. The system effectively parses, parameterizes, and classifies the Reuters corpus documents. Based on the evaluation tests, the performance of the various combinations is summarized in Table 1.

Table 1: Final Accuracy Results

Classifier	**Binary**	**TF**	**TF-IDF**
Minimum Distance	71.74%	50.33%	76.06%
Naive Bayes	75.20%	77.76%	**82.91%**
KNN ($k = 5$)	**82.38%**	75.37%	78.96%

The project demonstrates that TF-IDF significantly boosts probabilistic models, while geometric models like Minimum Distance are highly sensitive to noise in Term Frequency. Finally, the bonus component showed excellent results compared to `scikit-learn`.